LLD Notes — Phase 3, Part 6

Composite · Proxy · Flyweight · Jumbled round 4 (Shala) | Stories, your doubts, your mistakes, interview answers

> **Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.**

Contents

26. Composite — the Diwali hamper · boxes inside boxes · exercise 30 (Daftar) · it must stay a tree

27. Proxy — Shinde kaka, the principal's peon · guard, remember, delay · exercise 31 (Tijori) · a cache needs a rule for forgetting

28. Flyweight — the wedding card printer · one stencil, 2,000 cards · exercise 32 (Naksha) · the shared part never changes

29. Jumbled round 4 (Shala) — seventeen requirements, all fifteen patterns · patterns working together

30. Cheat sheet for all fifteen patterns · look-alikes · new rules · mistake tracker · recall questions

26. Composite

The story: the Diwali hamper

A sweet shop on Laxmi Road sells Diwali hampers. One customer orders boxes inside boxes, with items at the ends:

```
Big hamper
├── Kaju katli            Rs 600
├── Chakli packet         Rs 150
└── Small gift box
    ├── Dry fruits        Rs 500
    ├── Diya set          Rs 120
    └── Tiny pouch
        └── Elaichi       Rs 80
```

> **Your answer: Rs 1,450.** Correct. And look at *how* you got it: the small gift box had no price of its own, so you **went inside it** and added what you found, then did the same for the tiny pouch. One method, used at every level. That is the pattern.

Why Sahil's first code failed

```
class Item { int price; }
class Box  { List<Item> items; }          // a box holds items

bigHamper.items.add(smallGiftBox);        // error: Box cannot be converted to Item
```

> **You said: "it should give me a compilation error, right?"** Yes. A `Box` is not an `Item`, so the code can't even describe the hamper. Sahil's second try, two lists (`List<Item>` and `List<Box>`), compiles. But **you** spotted its problem: every piece of code must check "what type is this?" and loop twice. When code has to ask an object's type before deciding what to do, the objects should be answering the question themselves.

The fix: one interface for both

```
interface Gift { int price(); }

class Item implements Gift {
    private final int price;
    Item(int price) { this.price = price; }
    public int price() { return price; }                // I just know my number
}

class Box implements Gift {
    private final List<Gift> contents = new ArrayList<>();   // items OR boxes
    void add(Gift g) { contents.add(g); }
    public int price() {
        int total = 0;
        for (Gift g : contents) total += g.price();     // ask each one; don't care what it is
        return total;
    }
}
```

```
bigHamper.price()
  → Kaju katli            → 600
  → Chakli                → 150
  → smallGiftBox.price()
        → Dry fruits      → 500
        → Diya set        → 120
        → tinyPouch.price()
              → Elaichi   → 80
        ← 700
← 1450
```

> **Composite:** a single thing and a group of things share one interface, so a group can hold other groups, to any depth. **One list, one loop, no type checks.**

You already use it: folders on your laptop. A folder holds files and other folders; "Properties" works out its size by asking everything inside it.

Quick win ✓ — "Rs 30 packaging for every box." Only `Box` changes: `return total + 30;` . It applies itself at every level. Three boxes, so 1,450 + 90 = **Rs 1,540**, with no code that counts boxes.

Exercise 30 — Daftar (an HR tool)

Part A, Q1: 6 out of 6.

Req	What it said	Answer	You
R1	Salary cost of any unit: company, department, team, one employee	Composite	✓
R2	Headcount of any unit	Composite, one more method	✓
R3	Junior, Senior, Lead: different bonus formulas, changed yearly	Strategy	✓
R4	Canteen bill: sum of dishes; a dish never contains dishes	Nothing: a list and a loop	✓
R5	Announcement to any unit; every employee inside is emailed	Composite, one more method	✓
R6	On joining, tell payroll and IT; joining stands if one fails	Observer	✓

Two good reasons: on R2, "a single employee returns one", which is the heart of the pattern; on R3, you considered Template Method and rejected it because there are no shared steps.

R1 — four classes, or two? You planned Company, Department, Team and Employee. The first three all do the same thing (loop and add) and differ only in their name. **Differ by data → one class with fields**, the Kharcha rule again. So: `Employee` and `Group` . Name the interface `OrgUnit` , not `Employee` : a department isn't an employee.

Q2 ✓ — the empty Marketing department. Zero and zero, no special case. The reason is the loop: the total starts at 0 and the loop runs zero times. One condition: the list must start **empty**, never `null` (`= new ArrayList<>()`), or the loop crashes.

Q3 ✓ — Engineering added inside Backend, which is inside Engineering. You corrected yourself from "compile error" to "runtime error, an infinite loop." The exact error is a `StackOverflowError` : each call waits for the next until Java runs out of room. The fix is to refuse at the door:

```
void add(OrgUnit unit) {
    if (unit == this || unit.contains(this))
        throw new IllegalArgumentException("Cannot add " + unit.name() + " to " + name);
    members.add(unit);
}
```

A Composite must be a tree. If something can end up inside itself, every method that walks the tree loops forever. Your laptop refuses to drag a folder into its own subfolder for the same reason.

Q4 ✓ — `add()` only on `Group` . Nothing goes inside an employee. On the interface, `Employee` could only throw: a method that exists but must never be called. The trade-off: code that builds the tree must know it holds a `Group` . That's fine, because building happens in one place.

Your Part B — the pattern was right; the text wasn't

✓ One list of `OrgUnit` , one loop, no type checks. `Employee` gave the simple answers (salary, 1, false), `add()` checked both cases, and the costs, headcounts and empty department all matched.

1. `public interface OrgUnit` . A file can hold only one public type, and it must match the file name (`Main`). The rule applies to interfaces as well as classes.
2. Five lines off by one space each. `"Email to Asha : ..."` had an extra space before the colon, and `"Cannot add Engineering toBackend"` was missing one.

Part C

Q1 ½ — "the highest salary in any unit." You planned a map inside `Group` . No map is needed: each unit answers for itself, like each box did. The honest cost of the pattern: a new question touches **three** places.

```
interface OrgUnit { ... int highestSalary(); }                    // 1. the new question
class Employee { public int highestSalary() { return salary; } }   // 2. my own salary
class Group {                                                      // 3. the biggest answer inside me
    public int highestSalary() {
        int max = 0;
        for (OrgUnit m : members) max = Math.max(max, m.highestSalary());
        return max;
    }
}
```

Your doubt: "won't this give the highest salary of all the groups mixed up?" No. A group only looks at its own members. `frontend.highestSalary()` is 80,000 (Meera), even though Asha earns 90,000, because Asha isn't inside Frontend. You get the answer for the thing you ask, like the small gift box answering 700, not 1,450. A map belongs to a **report** that walks over the units and asks each one.

Every Composite method has the same shape: the single thing answers for itself; the group combines its members' answers. Only one line differs between `salaryCost()` and `highestSalary()` .

Q2 ✓ — the email to Ravi throws. As written, Meera gets nothing. Your fix: try/catch around each member's `announce()` , log it, carry on. Note the contrast: one failed email doesn't make the others wrong, so this is the "carry on" use of catch; a failed payment makes the booking wrong, so there you let it throw. Better still, return a summary: "sent to 2 of 3, failed: Ravi."

Q3 ✓ — "Why not one flat list with a department field?" The concession came first, then the reason (teams inside departments) and when flat wins. Sharper: the real problem is that **the depth isn't fixed**, so every new level would need a new field and new filters.
Script: "You'd be right if the structure were flat, one department per employee; I'd use a list and a filter there. Here units nest inside units and the depth can change, so I'd use a Composite: any unit answers for itself, however deep the tree goes."

27. Proxy

The story: Shinde kaka, the principal's peon

At a Pune college, the principal sits in an inner office. Shinde kaka sits at a desk outside, and nobody reaches the principal except through him. One morning, a man with no ID asks to see the principal; ten students ask, one after another, "When do the exams start?" (the principal told kaka at 9 am); and the principal isn't even in yet.

Your guess	What it shows
Kaka turns the man away; the principal never knows ✓	the check happens **before** the real one is involved
Kaka answers all ten himself ✓	he doesn't disturb the principal for an answer he already has
"Students are more formal with the principal"	true in life, but the *question* is the same. In code the stand-in has **exactly the same methods**, so the caller can't tell

```
interface Principal { String answer(String question); }

class Peon implements Principal {                // same interface as the principal
    private final Principal real;
    Peon(Principal real) { this.real = real; }

    public String answer(String question) {
        if (!hasIdCard()) throw new AccessDeniedException("No ID, no entry");   // the check
        return real.answer(question);            // otherwise, pass it through
    }
}
```

Kaka's three jobs

Job	Kaka	In code	In Spring
Guard	turns away the man with no ID	check, then pass on	`@PreAuthorize`
Remember	answers the repeated question himself	a map; skip the real call	`@Cacheable`
Delay	calls the principal in only when needed	create on first use	Hibernate lazy loading

You already use it every day: `@Transactional` doesn't change your class. Spring wraps your bean in a proxy with the same methods, which opens a transaction, calls your method, then commits or rolls back. (You noted that `@PreAuthorize` is aspect-oriented: Spring's AOP works through proxies.)

Quick win (c) — lazy loading is *delay*, not *remember*. `findById(7)` loads the order but not its items; a stand-in sits where the list should be. Only `order.getItems()` hits the database, and if your code never calls it, the items are never fetched.
Remember saves doing the same work **twice**. **Delay** saves doing work that may **never** be needed.

Decorator vs Proxy — the same shape in code

Your guess: (1) the proxy's input varies while the decorator's doesn't; (2) a decorator builds combinations, while a proxy stands in for the actual service. (1) was a coincidence of the examples: the Katta decorators took `process(String message)`. (2) is the real difference.

	Decorator	Proxy
What it's for	**adds** to the result	**controls** access to the real thing
Always calls the inner one?	**yes**, and uses its answer	**not always**: may refuse, or answer from memory
Does the caller know?	yes, it picks the layers	usually not
How many?	several, stacked	usually one per job

A decorator adds. A proxy controls. The test: **adding or controlling?** (For Decorator vs Chain it was "adding or deciding?")

Exercise 31 — Tijori (a CA firm's document vault)

Part A, Q1: 6 out of 6, with the structure explained each time.

Req	What it said	Answer	You
R1	Only partners may open a client's tax file	Proxy: guard	✓
R2	A fetch takes 4 seconds; the same file is opened often	Proxy: remember	✓
R3	500 files listed; each 20 MB scan loads only on click	Proxy: delay	✓
R4	Watermark, password, compress, in any combination	Decorator	✓
R5	SDK's `getDoc(int)` returns bytes; we call `open(String)`	Adapter	✓
R6	Every file has a name and an upload date	Nothing: a record	✓

```
class LazyScan implements Scan {          // R3: the dashboard holds 500 of these, each tiny
    private final String fileId;
    private Scan real;                    // empty until needed
    public byte[] bytes() {
        if (real == null) real = archive.loadScan(fileId);   // first click: load it now
        return real.bytes();
    }
}
```

Q2 ✓ — the map checked before the role. Role first, always. In the wrong order the junior gets the file straight from the map: no crash, just a tax file shown to the wrong person. Cleaner still, use **two small proxies**, so the wiring fixes the order:

```
DocumentVault vault =
    new GuardProxy(               // outside: checks the role first
        new CachingProxy(         // inside: remembers files
            new ArchiveVault())); // the real one
```

Check permission before anything else, including the cache.

Q3 ½ — file F corrected on the server; the map holds the old copy. You named the problem exactly (**stale data**) and said yourself your fix wasn't the real one. A fixed-size map that drops the least recently used entry limits *memory*. But F was opened a minute ago, so it's the last entry that would be dropped. Size answers "how much do I remember?"; this question asks "when do I stop trusting it?"

```
public void update(String id, Document doc) {   // fix 1: forget it when it changes
    real.update(id, doc);
    cache.remove(id);
}
// fix 2: forget it after a while (an expiry time), for edits made where we can't see them
```

A cache needs a rule for forgetting. The principal moves the exam date at noon. Either he tells kaka straight away (fix 1), or kaka re-checks every hour (fix 2). A kaka who does neither gives out the wrong date all day.

Q4 ✗ — a junior is refused. You said: "log a good message", not null, not throw. The motive (the junior should know why) is right. But a log goes to a server file the junior never sees, and `open()` must still return *something*, which can only be `null` or an empty document.

Option	What the caller sees
return `null`	nothing wrong… until a `NullPointerException` somewhere else
return an empty document	"this file is blank", which is untrue
log, then return	one of the two above
throw	a clear stop, with the reason attached

The code throws; the screen decides how to say it kindly. The proxy throws `AccessDeniedException`; the layer near the screen catches it and shows "You don't have access to this file. Please ask a partner." In Spring Boot, a `@ControllerAdvice` handler turns it into a 403 with a readable message.

Your Part B — compiled first time, exact output

All 11 lines matched, text included. Both proxies had the same shape. The guard threw (no null, no log) and covered both `open` and `update`; the cache forgot on update; the wiring order was right.

```
  [archive] fetching F1
1. Sharma tax 2025
2. Sharma tax 2025                ← no fetch: remembered
3. Ravi is not a partner          ← no fetch: stopped at the door
  [archive] updated F1
  [archive] fetching F1           ← the update made the cache forget
4. Sharma tax 2025 (corrected)
```

One tidy-up: the role check was written twice. Pull it into one `private void check(User user)`, so a rule change ("partners and managers") can't update one copy and miss the other.

Part C

Q1 ½ — record every open, including cached ones and refusals. You saw the real difficulty: a cached open never reaches the archive. But a list inside `ArchiveVault` that the cache must call makes the cache do two jobs, and **refused attempts are still missed**. (Wording: the proxies aren't "child classes" of the archive; they *hold* one. Has-a, not is-a.)

```
DocumentVault vault =
    new AuditProxy(             // outermost: sees EVERY attempt
        new GuardProxy(
            new CachingProxy(
                new ArchiveVault())));
```

Audit proxy placed…	Sees refusals?	Sees cached opens?
outermost	yes	yes
between guard and cache	no	yes
next to the archive	no	no

To see everything, stand at the front door. A proxy only sees the calls that reach it.

Q2 mostly ✓ — two partners open F3 at the same instant. You said `ConcurrentHashMap`. Both check, both find nothing, both fetch, both put. The double fetch is **waste** (the same file twice); a plain `HashMap` written by two threads at once is a real **bug**. But a thread-safe map makes each call safe, not two calls in a row. It's check-then-act again:

```
public Document open(User user, String id) {
    return map.computeIfAbsent(id, key -> vault.open(user, key));   // check + fetch + store, one step
}
```

Q3 ✓ — "Put the role check and the map inside ArchiveVault. Why three classes?" The concession came first, then separation of concerns with your own examples (more roles, different kinds of logging). Two more points: each proxy can be switched off in the wiring line (no cache in tests), and each is reusable in front of any vault.
Script: "You'd be right for something small, and I'd start with it in one class. I'd split it once those rules start changing separately: access rules change with the org, caching with performance, auditing with compliance. As separate proxies, each can be changed, tested or removed without touching the vault."

28. Flyweight

The story: the wedding card printer

A press in Appa Balwant Chowk prints 2,000 invitations for a family in Kothrud: the same Ganesh motif, gold border and verse on every card, with only the guest's name and address changing. Sahil's class kept the 5 MB artwork inside every `Invitation`, and the program crashed.

Your guess	Result
2,000 × 5 MB = 10,000 MB, so it crashes ✓	about 10 GB
The same on every card: address, artwork, verse	½: the address is the **guest's** address, so it differs. Artwork and verse are the same.
"A stencil, used for all of them" ✓	yes, with one correction: **one** stencil, not 2,000 copies of it

```
class CardDesign {                  // the stencil: made ONCE
    final byte[] artwork;           // 5 MB
    final String verse;
}

class Invitation {                  // tiny: made 2,000 times
    final String guestName;
    final String address;
    final CardDesign design;        // points to the shared stencil; doesn't copy it
}
```

	Sahil's version	Shared design
Artwork in memory	2,000 copies	**1**
Total	about 10 GB	**about 5 MB**, plus a few bytes per card

You already use it: Java's **string pool**. `String a = "Pune"; String b = "Pune";` creates one "Pune", and both variables point to it.

Quick win ✓ — `a.design.verse = "A new verse".` You noticed the field is `final`, so the line won't compile, and that if it could change, the Kulkarni family's card would change too. One refinement: with `final`, **nobody** can change it after it's created, not even its own class.

	Lives in	Example
Shared part: the same for many, never changes	the one shared object	artwork, verse
Own part: different for each	each small object	guest name, address

A shared object must never change after it's created. Anything that differs per card belongs on the card.

Exercise 32 — Naksha (a live map of Pune)

Part A, Q1: 5 out of 6.

Req	What it said	Answer	You
R1	50,000 autos, each with a position, one 2 MB icon	Flyweight	✓
R2	Four types with an icon each; type as text; load once	Flyweight + a factory that keeps them	½ (said Strategy + Factory)
R3	Number plate and driver's name	Nothing: fields	✓
R4	12 depots, a 1 MB description each	**Nothing**	½ (said a delay proxy)
R5	A route takes 3 seconds; asked for often	Proxy: remember	✓
R6	One settings object, the same everywhere	One shared instance, injected	✓

R1 — the picture. You said "create the rickshaw object once and only change the position." One rickshaw object would mean one rickshaw on the map. It's **50,000 small objects**, each with its own position, all pointing to **one icon**.
R2 — icons are data, not behaviour. Strategy is for a different *formula* per type; an icon is a picture. Your description of the factory was exactly right:

```
class IconFactory {
    private final Map<String, Icon> icons = new HashMap<>();
    Icon iconFor(String type) {
        return icons.computeIfAbsent(type, t -> loadIcon(t));    // load once, then reuse
    }
}
```

R4 — no pattern needed. A delay proxy was right in Tijori (500 files × 20 MB). Here it's 12 × 1 MB = 12 MB, which a phone holds easily. It isn't Flyweight either: each description is different.

Flyweight needs two things: a great many objects, **and** heavy data that is the same across them. If either is missing, you don't need it.

Q2 ✓ — Flyweight vs a caching proxy. You gave two real differences: a flyweight never changes, while a cached answer can go stale; and a proxy stands in front of a service, while a flyweight is just shared data. The quickest test is a third one:

	Flyweight	Caching proxy
Saves	**memory**	**time**
The problem	too many objects holding the same heavy data	the same slow work done again and again
Clue in the requirement	"50,000 of them"	"takes three seconds"

Q3 ✓ — `bus.icon.colour = RED` for the selected bus. Every bus turns red, because they share one icon. Your option 1 is the answer: "selected" belongs to the bus. (Option 2, a wrapper per vehicle around the shared image, is option 1 under another name; a mutable field on the *shared* object is the bug.)
The test for any field: the same for all of them, forever → the shared object. Differs, or can change → the small object.

Your Part B — the pattern was right; one line broke

✓ **The factory created each icon once and returned the same object after that. Only two icons loaded, and `v1 and v4 share one icon: true`.**

1. Compile error: "String cannot be converted to boolean".

```
return icon.type() + (plate + " (" + x + ", " + y + ") " + selected ? " [SELECTED]" : "");
```

In Java, `+` is done before `? :`. Java glued everything up to `selected` into one String, then tried to use it as the yes/no test. Bracket the choice: `+ (selected ? " [selected]" : "")`.
2. Then five text differences on each line: no space after the type, the word `at` missing, an extra space after the comma, a trailing space, and `SELECTED` in capitals.

```
// AUTO MH12-1001 at (11,21) [selected]        ← paste the expected line as a comment first
return icon.type() + " " + plate + " at (" + x + "," + y + ")" + (selected ? " [selected]" : "");
```

3. A separate `cnt` counter. The map already knows: `return map.size();`. **Compute it, don't store it**: two things that must agree will one day disagree.

Part C

Q1 nearly — bikes in three colours. Three icon objects for 20,000 bikes ✓. But a key of only the colour would hand a future red *bus* the red *bike* icon. The key is `type + "-" + colour`, for example `"BIKE-RED"`. **The key must include every detail that changes the shared object.**

Q2 ✓ — a hundred vehicles ask for "AUTO" at once. Both halves of the fix, `ConcurrentHashMap` plus `computeIfAbsent`, reused from Tijori without a hint. Why it matters more here: two vehicles could end up with two different icon objects. The sharing quietly fails, `v1.icon() == v4.icon()` is false, and there could be a hundred copies of the 2 MB icon.

Q3 ✓ — "Memory is cheap now. Why bother?" All three moves: the concession, scale and performance, and "fine for a small app." Sharper: **use the number** and **name the hidden cost**, which is garbage collection.
Script: "You'd be right for a few hundred objects, and I wouldn't use it there. Here it's 50,000 vehicles with a 2 MB icon each, which is 100 GB against 2 MB. And fewer objects means less garbage collection work, so the app stays smooth."

29. Jumbled round 4 — Shala

School management software for a group of Pune schools. All fifteen patterns in play, no hints, three requirements that need none. You answered after a three-day gap.

Q1 — you scored 17 out of 17

Req	What it said	Answer	You
R1	Roll number, name, date of birth	Nothing: a record	✓
R2	Leave: class teacher to 2 days, principal to 7, trustee above	Chain of Responsibility	✓
R3	3,000 report cards, one 3 MB crest and letterhead	Flyweight	✓
R4	Timetable editor: move, swap, delete; Undo and Redo	Command	✓
R5	Fee unpaid 15 days after the due date: remind the parent	**Nothing: a scheduled job**	✓
R6	Wings, classes, sections, students: pending fees of any of them	Composite	✓
R7	Portal SDK's `pushStudent(Map)` vs our `register(Student)`	Adapter	✓
R8	Leave: DRAFT → SUBMITTED → APPROVED or REJECTED, with rules	State	✓
R9	Admission: five steps; office desk and parents' app	Facade	✓
R10	Export: watermark, signature, QR code, any combination	Decorator	✓
R11	On results: SMS, notice board, ranks; one may fail	Observer	✓
R12	Concession by category, as text; rules change yearly	Strategy + Factory	✓
R13	Only the class teacher may see the medical record	Proxy: guard	✓
R14	Exams: the same four steps; evaluation and weightage differ	Template Method	✓
R15	Field trip: 2 required, 9 optional, a date rule, fixed once approved	Builder	✓
R16	"Next year we might add a hostel module"	Nothing: YAGNI	✓
R17	One academic calendar everywhere	One shared instance, injected	✓

R5 — got it on the fourth appearance. Missed with the 7-day ticket, the unscanned PuneGo ticket and Jhatpat's rider; answered straight away here. **Sharpenings:** run the job **every day** (with a 15-day gap, some reminders arrive a month late) · R6's wing, class and section are one `Group` class · for R17, say the whole sentence: "one instance, created once and injected."

Q2 ½ — Chain and State together on a 5-day leave

Two things got merged. (1) Draft, submit and approval all ran in one facade call. In real life they are separate moments, by different people. (2) You treated "nobody in the chain could approve" as a rejection. A rejection is a **person's decision**; "nobody can approve" is an error in the setup. The chain decides **whose desk** the application goes to.

Moment	Who	What happens	Pattern
1. Fills the form	teacher	a leave is created in **DRAFT**	State
2. Clicks Submit	teacher	DRAFT → **SUBMITTED**. The chain: class teacher (2)? no. Principal (7)? **yes**. Sent to the principal.	State, then Chain
3. Next day	principal	Approve → **APPROVED**, or Reject → **REJECTED**	State
4. Tries to withdraw	teacher	the APPROVED state refuses and throws	State

```
// moment 2
leave.submit();                                  // State: only a DRAFT can be submitted
String approver = approvalChain.approve(leave);  // Chain: returns "Principal"
leave.assignTo(approver);

// moment 3: a separate call, the next day
leave.approve(currentUser);                      // State: only a SUBMITTED leave can be approved
```

The Chain answers "who decides?" The State answers "what is allowed right now?" The person makes the decision.

Q3 ✓ — Flyweight and Decorator together

A watermark drawn onto the shared crest appears on **all 3,000** report cards. And your second point: if every card always needs it, it isn't optional, so it belongs in the base design (the Katta rule).

```
Watermark( Signature( ReportCardExport ) )
                        │
                        └── points to → shared crest   (read, never written)
```

Decorate the copy, never the shared original. The flyweight never changes; the decorator adds layers to one card's output.

Q4 ✓ — Strategy (R12) vs Template Method (R14)

	R12 concession	R14 exams
What differs	the **whole** calculation	**two steps** in a fixed order
An order to protect?	no	**yes**: set, conduct, evaluate, publish
When the type is known	at **run time**, as text	when the exam is created
Built with	**composition**: pick and swap an object	**inheritance**: the parent's `final` method

The deciding question: is there a shared skeleton to protect? Yes → Template Method. No → Strategy.
Script: "Both differ by type, but by different amounts. A concession is one whole formula per category, picked at run time from the text and replaced every year, so each is a strategy object. An exam is a fixed four-step process whose order must never change, and only two steps differ, so a parent class owns the order in a final method."

30. Consolidation

Look-alikes

Pair	The question that tells them apart
Decorator vs Proxy	Adding, or controlling?
Decorator vs Chain	Adding (every link runs), or deciding (a link can stop it)?
Flyweight vs caching proxy	Saving memory, or saving time?
Flyweight vs one shared instance	One shared object per kind (an icon per vehicle type), or exactly one for the whole system?
Composite vs a plain list	Do items contain items, to any depth?
Template Method vs Strategy	Is there a shared skeleton to protect?
Chain together with State	Who decides, vs what is allowed right now
Observer vs scheduled job	Did something happen, or did something *not* happen?
Observer vs required step	If it fails, is the operation still valid?
Adapter vs Decorator vs Facade	One thing, change shape · one thing, add behaviour · many things, one front door

Requirement → pattern cheat sheet (all fifteen)

What the requirement sounds like	Answer
"a different formula per type", "we change the rules every year"	**Strategy**
"the type arrives as text and we must pick the right object"	**Factory**
"must be the same everywhere", "one source of truth"	**one shared instance**, injected
"2 required, many optional, a rule across fields, never changes after"	**Builder**
"when X happens also do A, B, C" and "X still counts if one fails"	**Observer**
"allowed in some statuses, refused in others"	**State**
"several extras can apply together, in any combination"	**Decorator**
"their SDK doesn't match our interface, and we can't edit it"	**Adapter**
"undo", "redo", "do it later", "retry"	**Command**
"up to X this person, up to Y that person, above that someone else"	**Chain of Responsibility**
"the same steps in the same order; only one or two steps differ"	**Template Method**
"several steps in order, and several callers need it"	**Facade**
"the total of any unit: the whole, a part, or one item", groups inside groups	**Composite**
"only X may…" · "slow, and asked for often" · "load it only when clicked"	**Proxy**: guard · remember · delay
"thousands of them, all with the same heavy picture or data"	**Flyweight**
"no A, no X" (payment before the order)	**nothing**: a required step, called directly
"X hasn't happened in N days"	**nothing**: a scheduled job, run every day
"always the same few fields"	**nothing**: fields, or a record
"a list whose items never contain items"	**nothing**: a list and a loop
"a dozen items, a megabyte each"	**nothing**: just load them
"one call to one class"	**nothing**: call it directly
"we wrote it ourselves; it needs a better name"	**nothing**: rename it
"maybe next year"	**nothing**: YAGNI

New rules from this group

Area	Rule
Composite	A single thing and a group share one interface, so groups can hold groups, to any depth.
Composite	Every method has the same shape: the single thing answers for itself; the group combines its members' answers.
Composite	A new question means a new method on the interface and in every class. That's the cost.
Composite	It must stay a tree. Refuse a group that would end up inside itself, at the moment of adding.
Composite	Start the list empty, never null; the empty group then handles itself. `add()` goes only on the group.
Proxy	Same interface as the real thing; it controls access. A decorator adds; a proxy controls.
Proxy	Three jobs: guard, remember, delay. One small proxy per job; the wiring fixes the order.
Proxy	Check permission before anything else, including the cache.
Proxy	To see every call, stand at the front door.
Cache	It needs a rule for forgetting: remove on update, or expire after a time.
Cache	A size limit answers "how much do I remember?", not "when do I stop trusting it?"
Concurrency	A thread-safe map makes each call safe, not two calls in a row. Use `computeIfAbsent`.
Flyweight	Needs many objects **and** heavy data that is the same across them. Move the heavy part into one shared object.
Flyweight	The shared object never changes after it's created. Anything that differs or changes lives on the small object.
Flyweight	The factory's key includes every detail that changes the shared object.
Weaving	Decorate the copy, never the shared original.
Weaving	The Chain answers who decides; the State answers what is allowed now; the person decides.
Failure	Refused means throw. The screen layer turns it into a friendly message.
Failure	In a broadcast, one failed member shouldn't stop the others: carry on, and report a summary.
Java	`+` is done before `? :`, so bracket the choice.
Java	One public type per file (class or interface), matching the file name.
Design	Compute it, don't store it: `map.size()`, not a separate counter.
Interview	Use the number: 50,000 × 2 MB = 100 GB against 2 MB.

Mistake tracker

Mistake	Status after this group
Observer where a scheduled job is needed	**fixed: Shala R5, on its fourth appearance**
Not conceding on "defend it"	**fixed: concession first in all three Group 5 defend questions**
Reaching for Bill Pugh	**gone: "one shared instance" in Naksha R6 and Shala R17**
Exact output text	**the main gap: two spacing slips in Daftar, five differences in Naksha; exact in Tijori**
Refused → return or log instead of throw	said "log it" in Tijori Q4; wrote `throw` correctly in Part B an hour later
A second `public` type in one file	**back again: `public interface` in Daftar**
Storing a value that can be computed	**back again: the separate `cnt` in Naksha**
Strategy for something that is data, not behaviour	new: Naksha R2 (icons)
A pattern for a case too small to need it	new: Naksha R4 (12 MB of depots)
Separate moments merged into one call	new: Shala Q2 (submit and approve)
Operator order, `+` before `? :`	new: Naksha `draw()`

Where you stand. **Recognition:** Daftar 6/6 · Tijori 6/6 · Naksha 5/6 · Shala **17/17** across all fifteen patterns, after a three-day gap.
Code: Tijori compiled first time with exact output; in Daftar and Naksha the pattern was right and only the text was off.
Habit that appeared: reusing a lesson unprompted (`computeIfAbsent` from Tijori, used in Naksha).
Two things to practise: paste the expected output line as a comment above your `return` and build the string to match it; and when something is refused, the method throws.

Recall questions — cover the answers

1. The hamper: what did you do when you reached the small gift box, and why is that the pattern?
2. Why did `List<Item>` fail, and what was wrong with two lists?
3. Write the `price()` method of `Box`.
4. Rs 30 per box: which class changes, and what is the hamper's new total?
5. Company, department, team, employee: how many classes, and why?
6. The empty Marketing department: what does it return, and what one condition makes that work?
7. A group ends up inside itself: what error, and where do you stop it?
8. Should `add()` be on the interface? Why not?
9. Write `highestSalary()` for `Employee` and `Group`. How many places change?
10. `frontend.highestSalary()`, when Asha (90,000) is in Backend and Meera (80,000) is in Frontend?
11. One email in an announcement fails: throw or carry on? Why is that different from a failed payment?
12. "Why not a flat list with a department field?" Give the script.
13. Kaka's three jobs, with a Spring feature for each.
14. Why must a proxy have exactly the same interface as the real object?
15. Remember vs delay: what does each save you from?
16. Decorator vs Proxy: the one-word test, and which one doesn't always call the inner object?
17. How does `@Transactional` work without changing your class?
18. Guard and cache: which check comes first, and what goes wrong the other way round?
19. Write the wiring line for the guard, the cache and the archive.
20. Why doesn't a fixed-size (LRU) map fix stale data?
21. Two ways for a cache to forget, and when to use each.
22. A junior is refused: null, empty document, log, or throw? Who shows the friendly message?
23. Where does an audit proxy go to see refused and cached opens?
24. Two threads miss the cache at once: what is waste, what is a bug, and what's the one-line fix?
25. Why isn't `ConcurrentHashMap` alone enough for "check, then store"?
26. "Why three classes for one vault?" Give the script.
27. The wedding cards: which fields are shared, and which belong to each card?
28. Why must the shared object never change after it's created?
29. Where does Java already use a flyweight?
30. 50,000 autos and one icon: how many objects of each?
31. Why isn't Naksha R2 (an icon per vehicle type) a Strategy?
32. 12 depots at 1 MB each: which pattern, and why?
33. Flyweight vs caching proxy: what does each save?
34. Where does "selected" live, and what happens if it's on the shared icon?
35. What must the factory's key contain for bikes in three colours?
36. Why does a race in the icon factory break the pattern itself?
37. "Memory is cheap." Give the script, with the number.
38. Why is `"a" + b + flag ? x : y` a compile error, and what fixes it?
39. A 5-day leave: which pattern finds the approver, which tracks the status, and who decides?
40. The report card watermark: why not draw it on the crest? Where does it go?
41. Shala R12 vs R14: what question decides Strategy or Template Method?
42. Name all fifteen patterns, with one trigger phrase each.